

Deep Dive: Object-Oriented Programming

CMPT 211: Introduction to Software Development

Bernal Manzanilla Saavedra

Concordia University of Edmonton

Winter 2026

The Limitations of Procedural Programming

So far, we have written **Procedural Code**: separating data (variables) from behavior (functions).

Why does Procedural Code break down in large systems?

- **Global State Sprawl:** As programs grow to thousands of lines, tracking which function modifies which list or dictionary becomes a nightmare.
- **Data Vulnerability:** If a list of sensor voltages is passed freely between 10 different functions, any one of those functions can accidentally delete or corrupt the data.
- **Lack of Real-World Mapping:** Procedural code reads like a list of computer instructions, making it difficult to model complex, real-world systems.

The OOP Solution: Encapsulated Entities

Object-Oriented Programming (OOP) solves these scalability issues through a fundamental shift in architecture.

Instead of asking: *"What sequence of mathematical steps must the computer take?"*

OOP asks: *"What entities exist in this system, and what can they do?"*

The Core Philosophy

We **bundle** the Data (State) and the Behavior (Methods) together into a single, cohesive, self-managing unit called an **Object**.

Definitions: The Class vs. The Object

1. The CLASS (The Blueprint)

- A **Class** is a theoretical template.
- It dictates the rules: "Any signal created from this blueprint *must* have a name and a sampling rate."
- It occupies very little memory because it holds no actual data.

2. The OBJECT (The Instance)

- An **Object** is the physical realization of the class in RAM.
- It holds concrete data (e.g., $f_s = 1000\text{Hz}$).
- We can stamp out millions of unique Objects from a single Class blueprint.

Terminology: Attributes and Methods

Once inside an object, standard programming terms change to denote **ownership**.

1. Attributes (The State)

Variables bound to an object. They represent what the object *knows* or *is*.

Example: A `Signal` object has attributes: `self.name`, `self.voltage_array`.

2. Methods (The Behavior)

Functions bound to an object. They represent what the object can *do*.

Example: A `Signal` object has methods: `self.filter_noise()`, `self.plot_data()`.

The Constructor: `__init__`

How does an object get its starting attributes the exact moment it is born?

```
class Signal:
    # __init__ is the Constructor Method.
    # It executes automatically upon creation.

    def __init__(self, sensor_name, fs):
        # We bind the incoming arguments to the object's State
        self.name = sensor_name
        self.sampling_rate = fs
        self.data = [] # We can also initialize empty states
```

Note: `__init__` uses double underscores (dunder), marking it as a special, built-in Python method that you should not call manually.

Demystifying self

The Golden Rule: The first parameter of every method must be self.

```
# 1. Instantiation (We only pass TWO arguments):  
my_sig = Signal("Heart_Rate", 1000)  
  
# 2. Behind the scenes, Python translates this to:  
Signal.__init__(my_sig, "Heart_Rate", 1000)
```

Why? Because self is the memory pointer. It tells the Class exactly *which* object's data to modify. Without self, the Class wouldn't know if it was updating my_sig or your_sig.

The Most Common Beginner Error

What happens if you forget to include `self` in a method?

```
class Signal:
    def __init__(self):
        self.data = [1, 2, 3]

    # ERROR: Forgot 'self' in the parameters!
    def print_data():
        print(self.data)

s = Signal()
s.print_data() # Crash!
```

TypeError: print_data() takes 0 positional arguments but 1 was given.

Python automatically tried to pass the object 's' in as the first argument, but your method wasn't prepared to catch it!

Pillar 1: Encapsulation (Data Hiding)

Definition: Encapsulation is the mechanism of restricting direct access to some of an object's components, and bundling the data with the methods that operate on it.

The Philosophy of the "Black Box":

- The outside world should not be allowed to directly touch an object's internal variables.
- If you want to change the `sampling_rate`, you must ask the object to do it via a **Setter Method**.
- **Why?** The Setter method can validate the data! It can reject a negative sampling rate, preventing the object from entering an illegal or corrupted state.

Encapsulation in Python: Convention vs. Enforcement

Unlike Java or C++, Python does not have strict private or public keywords. It relies on developer conventions.

```
class Signal:
    def __init__(self):
        # A single underscore prefix tells other developers:
        # "This is private. DO NOT touch this directly."
        self._fs = 1000

    # The authorized "Setter" method
    def set_sampling_rate(self, new_fs):
        if new_fs <= 0:
            raise ValueError("Sampling rate must be positive.")
        self._fs = new_fs
```

Note: A double underscore `__fs` invokes "name mangling" for stricter hiding, but a single underscore is the standard engineering practice.

Pillar 2: Inheritance (The "IS-A" Relationship)

Definition: Inheritance allows a new class (Child/Subclass) to absorb all the blueprints (attributes and methods) of an existing class (Parent/Superclass).

- **Why?** It enforces the **DRY Principle** (Don't Repeat Yourself).
- If we have a generic `Signal` class, and we want to create an `AudioSignal` class, we don't rewrite the array management code. We just inherit it, and then add audio-specific methods like `play_sound()`.
- We call this an **"IS-A" relationship**: An `AudioSignal` *is a* `Signal`.

Inheritance in Python: `super()`

How do we ensure the Parent's setup code runs when we build a Child object? We use the `super()` function.

```
class Signal: # PARENT
    def __init__(self, name):
        self.name = name
        self.data = []

class AudioSignal(Signal): # CHILD (inherits from Signal)
    def __init__(self, name, channels):
        # 1. Ask the Parent class to handle its part of the setup
        super().__init__(name)

        # 2. Handle the Child-specific setup
        self.channels = channels

# AudioSignal now has .name, .data, AND .channels!
track = AudioSignal("Vocal_Mic", 2)
```

Pillar 3: Polymorphism

Definition: "Poly" means many, "morph" means form. Polymorphism is the ability to present the same interface for differing underlying forms (data types or classes).

- In practice, it means you can have different classes that all feature a method with the **exact same name**.
- The program doesn't need to know exactly what type of object it is dealing with, as long as the object knows how to respond to the method call.

Polymorphism in Python: "Duck Typing"

Python uses a concept called **Duck Typing**: *"If it walks like a duck and quacks like a duck, it must be a duck."*

```
class AudioSignal:
    def process(self):
        print("Applying acoustic echo cancellation...")

class VideoSignal:
    def process(self):
        print("Applying color grading matrix...")

# A generic list holding different types of objects
media_streams = [AudioSignal(), VideoSignal(), AudioSignal()]

# POLYMORPHISM IN ACTION:
for stream in media_streams:
    # Python dynamically figures out WHICH process() to call
    # based on the object's true identity!
```

Questions?

Your next task: Lab 5
Designing the Encapsulated Signal Class.